Project Specs

This is the source of truth for architecture, data, and APIs. **Agree on the API contract and event schema in hour 1** so all three of us can build in parallel against mocks. If something here changes, update this doc.

System overview

```
Chrome extension (content script + service worker)
  |-- Gemini: caption alignment, grammar notes, weekly summary
  |-- ElevenLabs: Text-to-Speech, Scribe speech-to-text
  v
Small backend (FastAPI or Node)   <-- also hides API keys
  |-- caches TTS audio
  |-- writes / reads events
  v
Tiger Data (Postgres + TimescaleDB)
  ^
Dashboard web app
```

The backend does triple duty: **key proxy**, **audio cache**, and **Tiger Data gateway**. The extension never connects to the database directly.

Tech stack

Layer	Choice	Notes
Extension	Manifest V3, TypeScript, Vite + CRXJS-style plugin	Hot reload during development
Backend	FastAPI (Python) or Node/Express	Pick whichever the data owner is faster in
Database	Tiger Data (Postgres + TimescaleDB)	Hypertable, continuous aggregates, compression

Dashboard	Next.js or Vite + React, plus a chart library	Poll for live updates
LLM	Gemini API	Structured JSON output
Audio	ElevenLabs TTS and Scribe STT	Multilingual model for Spanish

Extension architecture

- **Content script:** detects the video, extracts captions, renders the overlay and sidebar, syncs to `video.currentTime`, handles the shadowing UI.
- **Background service worker:** makes calls to the backend, caches results in `chrome.storage.local`.
- **Prompt module:** the alignment prompt and JSON schema. This is the most important file in the project, so iterate on it early.
- **Options page:** target language, level, and backend URL. For the hackathon, keep it minimal.

Getting captions

1. **Primary:** read `ytInitialPlayerResponse` for the caption track URL and fetch it as `json3` (timestamps included).
2. **Fallback:** a `MutationObserver` on YouTube's on-screen caption element. No lookahead, so no pre-translation, but fine as a demo fallback.

Rendering

- Hide YouTube's native captions with CSS. Inject our own overlay positioned over the player.
- The sidebar is an injected panel (a plain injected div is faster to build than Chrome's side panel API).
- Each aligned pair gets a color from a fixed palette by index. Both the source and target lines use the same color per pair.

Translation strategy

Pre-translate the whole transcript instead of translating live. Calling an LLM per caption line adds latency and breaks sync.

1. When a video loads, pull the full transcript.
2. Send it to Gemini in batches of about 30 lines.
3. Translate the first batch immediately so captions start fast. Fill in the rest in the background.
4. Cache results (by video ID + language + level). Playback only looks up the current line.

Alignment spec

Don't align by word position, since word order differs across languages. Have Gemini align meaning and return structured JSON.

```
{
  "start": 12.4,
  "end": 14.1,
  "src": "how are you?",
  "tgt": "¿cómo estás?",
  "pairs": [
    {"src": "how", "tgt": "cómo", "note": "interrogative adverb"},
    {"src": "are you", "tgt": "estás", "note": "estar, 2nd person singular, present"}
  ]
}
```

Rules for the prompt:

- A pair can be a phrase, not just a word. This handles idioms and verb conjugations that don't map one to one.
- Every word in `tgt` should belong to exactly one pair, where possible.
- Include a short `note` for each pair (part of speech, tense, formality, idiom flag).
- Return JSON only, with no preamble or markdown fences. Parse defensively and strip fences if they appear.
- Keep the batch response small enough to return quickly. Validate that pair text actually appears in the lines, and fall back to uncolored text if not.

ElevenLabs integration

Feature	Product	Notes
Word and line playback	Text-to-Speech (multilingual model)	Cache audio by hash of text + voice + speed

Slow mode	TTS with speed setting	"Say it slowly" for hard lines
Shadowing transcription	Speech-to-Text (Scribe)	Verify Spanish accuracy and latency early
Voice tutor (stretch)	Conversational AI agent	Only after shadowing works

Scoring approach

1. Normalize both strings (lowercase, strip punctuation, optionally strip accents).
2. Run a word-level diff (Levenshtein or a standard diff library).
3. Per-word result: `correct`, `missed`, or `substituted`.
4. Overall score = matched words / total words.
5. Render the user's attempt with the same pair colors, with missed words underlined in red.

Describe this as "intelligibility scoring." It measures what Scribe heard, not true phonetic accuracy. It is a good proxy and perfectly demoable. Optional: ask Gemini for a short tip on why a word may have been misheard.

Database spec (Tiger Data)

Event hypertable

```
CREATE TABLE learning_events (
  time        TIMESTAMPTZ NOT NULL,
  user_id     TEXT NOT NULL,
  event_type  TEXT NOT NULL,   -- 'lookup','save','shadow_attempt','review','caption_view'
  video_id    TEXT,
  word        TEXT,
  lang        TEXT,
  score       DOUBLE PRECISION, -- shadowing score, review result
  meta        JSONB
);
SELECT create_hypertable('learning_events', 'time');
```

Relational tables (the "unified stack" story)

```
CREATE TABLE users (
  user_id    TEXT PRIMARY KEY,
```

```
  display_name TEXT,
  target_lang TEXT DEFAULT 'es',
  level      TEXT DEFAULT 'A2',
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE vocab (
  user_id    TEXT REFERENCES users(user_id),
  word       TEXT NOT NULL,
  translation TEXT,
  video_id   TEXT,
  video_ts   DOUBLE PRECISION,   -- timestamp where the word appeared
  status     TEXT DEFAULT 'new', -- new | learning | known
  saved_at   TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (user_id, word)
);
```

Continuous aggregates to build

- **Daily activity:** active minutes and words looked up per day
- **Daily shadowing score:** average score per day (the "accent improving" line)
- **Per-word score history:** which words keep getting missed
- **Hourly heatmap:** activity by day and hour

Performance features to use and mention

- **Compression policy** on events older than a day
- **Seed script** generating a few hundred thousand synthetic events so charts show real trends. Be upfront that the seed data is synthetic. Live demo data should flow in too.
- **Real-time view:** dashboard polls every 1-2 seconds so a fresh shadowing attempt appears live

API contract (draft)

All endpoints take and return JSON unless noted. Authentication is a simple shared secret or none for the hackathon.

Method	Endpoint	Purpose	Request	Response

POST	/align	Align a batch of caption lines	{ lang_src, lang_tgt, level, lines: [{start, end, text}] }	{ lines: [AlignedLine] }
POST	/explain	Sidebar explanation for a word or phrase	{ word, context, lang_src, lang_tgt }	{ gloss, grammar_note, example, formality }
POST	/tts	Text to speech	{ text, lang, speed }	audio (mp3)
POST	/stt	Transcribe a recording and score it	multipart: audio, target_text	{ transcript, score, words: [{word, status}] }
POST	/events	Log a learning event	{ user_id, event_type, video_id, word, lang, score, meta }	{ ok: true }
POST	/vocab	Save a word	{ user_id, word, translation, video_id, video_ts }	{ ok: true }
GET	/vocab	List saved words	?user_id=	{ words: [...] }

GET	/stats/score-trend	Daily average shadowing score	?user_id=&days=	{ points: [{day, avg_score}] }
GET	/stats/activity	Words looked up and minutes per day	?user_id=&days=	{ points: [...] }
GET	/stats/trouble-words	Lowest-scoring words	?user_id=&limit=	{ words: [{word, avg_score, attempts}] }
GET	/stats/heatmap	Day x hour activity	?user_id=	{ cells: [{dow, hour, count}] }
GET	/stats/recent	Latest events (for the live view)	?user_id=&since=	{ events: [...] }

Mock first. Person C should publish mock responses for every endpoint in the first two hours so the extension and dashboard can be built without waiting.

Event types

event_type	When it fires	Key fields
caption_view	A video's captions are loaded or a minute of watching passes	video_id, lang
lookup	User clicks a word or phrase	word, video_id

`save`	User saves a word to vocab	`word`, `video_id`
`shadow_attempt`	User records and gets scored	`word` or line, `score`, `meta.transcript`
`review`	Flashcard review (if built)	`word`, `score`

Dashboard spec

Small web app, five panels:

1. **Pronunciation trend:** average score per day (live-updating)
2. **Words learned vs. reviewed:** stacked daily counts
3. **Trouble words:** lowest average scores, each with a "practice" button that plays TTS
4. **Study heatmap:** day x hour
5. **Top videos by new vocabulary:** which content teaches you the most

Optional flourish: a Gemini-written weekly summary generated from a few aggregate rows ("You improved on rolled r's, but keep missing subjunctive forms").

Configuration and secrets

- Keep all keys in the backend's `.env`. Never commit them.
- Commit a `.env.example` with variable names only.
- Expected variables: `GEMINI_API_KEY`, `ELEVENLABS_API_KEY`, `DATABASE_URL`, `PORT`.
- The extension only knows the backend URL (set on the options page).

Things to verify in the first hours

- **ElevenLabs Scribe:** Spanish accuracy and latency, tested with a real recorded clip.
- **Microphone access from a content script:** `getUserMedia` inside a page's context can trigger a permission prompt per site. A common workaround is recording from an extension page or offscreen document. If this forces a redesign, find out early.
- **Tiger Data:** free tier limits and connection method from the backend.

- **Frame capture from YouTube** (only if doing the visual mode): canvas capture can hit cross-origin problems.
- **Sponsor mentors:** both ElevenLabs and Tiger Data usually staff booths. Talking to them helps, and judges often notice teams that engaged.

Open decisions

Decision	Options	Owner	Status
Backend language	FastAPI vs. Node	Person C	Open
Dashboard framework	Next.js vs. Vite + React	Person C	Open
Demo video	One English YouTube clip with clean manual captions	All	Open
Target language	Spanish (default)	All	Decided
Recording location	Content script vs. offscreen document	Extension lead	Open